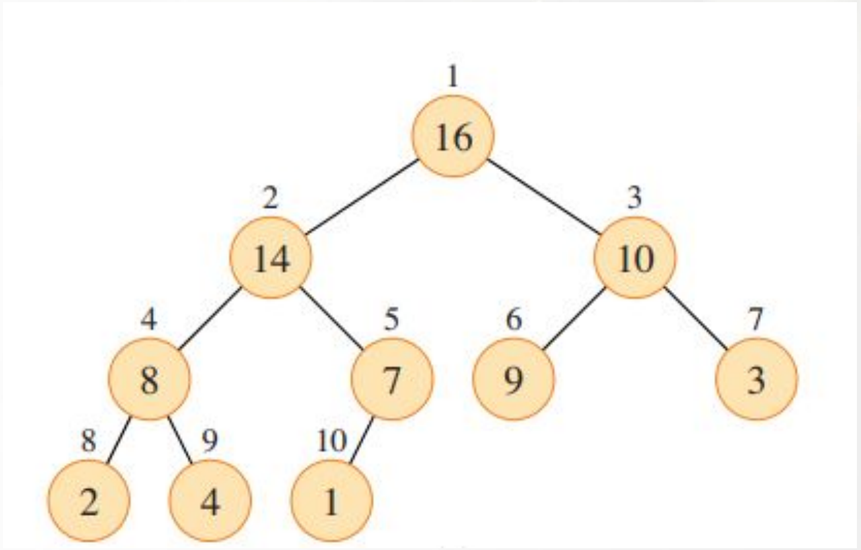
ALGORITHMS ANALYSIS AND DESIGN

Section 6: Heap Sort and Counting Sort

HEAPS

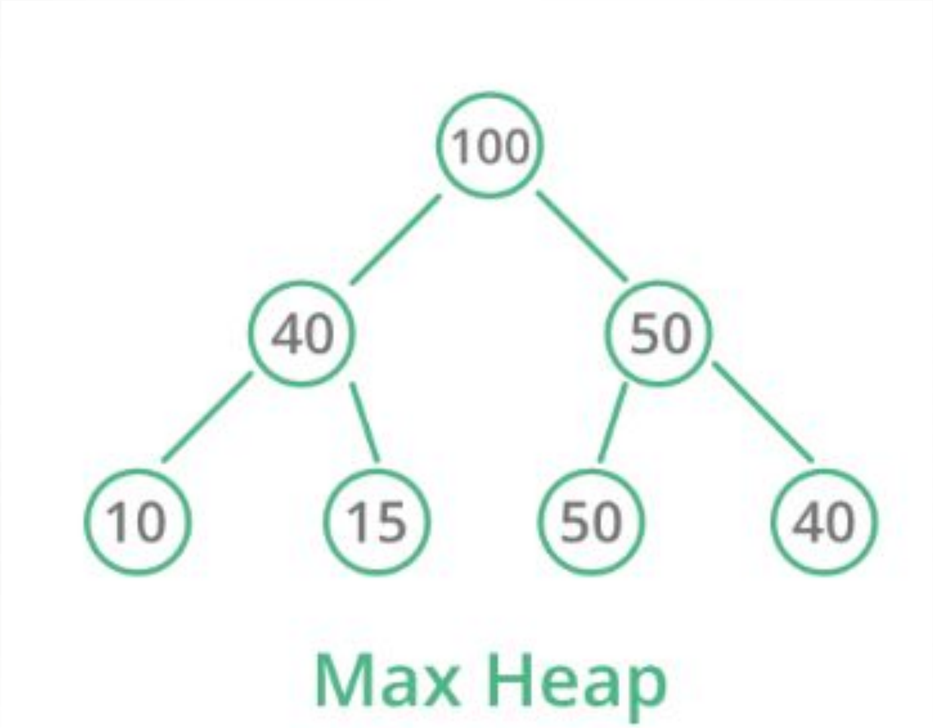
Binary Heaps

- **A Heap is a specialized tree-based data structure that satisfies two main properties:**
  - **Structural Property:** It is a complete binary tree — all levels are filled except possibly the last, which is filled from left to right.
  - **Heap Order Property:** The value of each parent node is ordered with respect to its children.
- **In simple terms:**
  - **A heap always keeps the “most important” element (largest or smallest) at the root.**



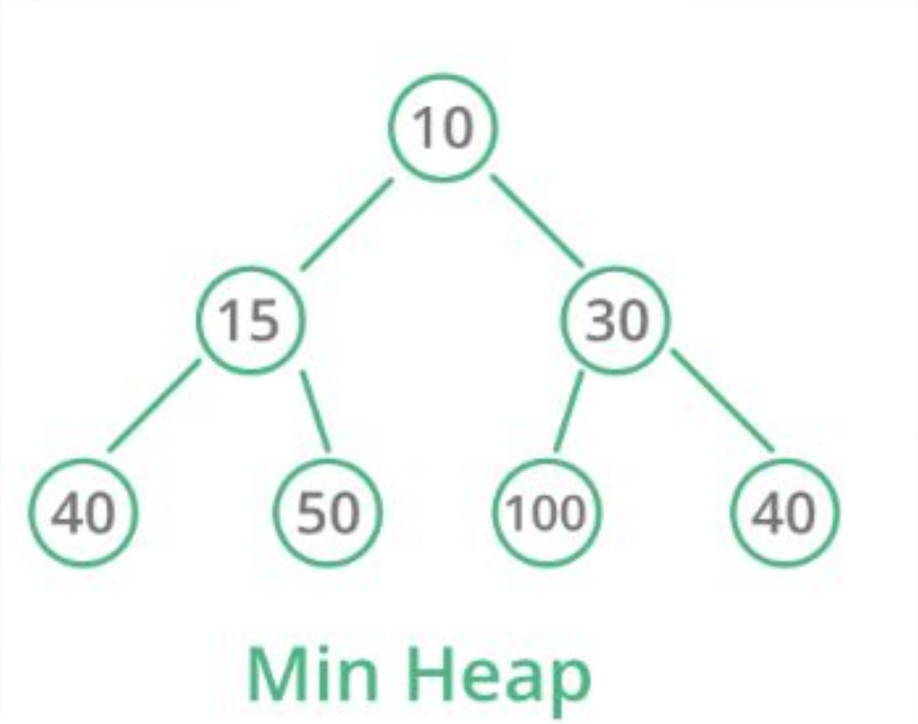
Binary Heaps - Max Heap

- Each parent node is greater than or equal to its children.
- The largest element is at the root.



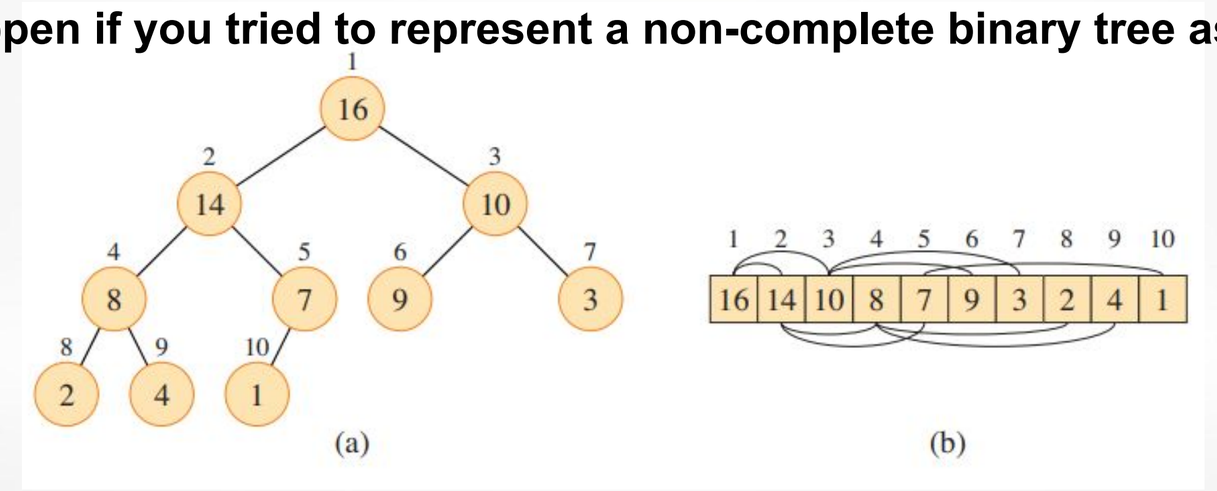
Binary Heaps - Min Heap

- Each parent node is less than or equal to its children.
- The smallest element is at the root.



Heap representation as an array

- Because a heap is a binary and complete tree, every level is fully filled except possibly the last, and all nodes are as far left as possible.
- This strict structure creates a ***fixed parent-child relationship*** based on indices, ***which allows the heap to be stored compactly in an array without any explicit pointers.***
- there's a simple way to compute the indices of its parent, left child, and right child
  - Parent(i) = Floor(i / 2)
  - Left(i) = 2i
  - Right(i) = 2i + 1
- **THINK What will happen if you tried to represent a non-complete binary tree as an array ?**



Sheet 5 – Question 1

- Is the array with values $\langle 33, 19, 20, 15, 13, 10, 2, 13, 16, 12 \rangle$ a max-heap?

Solution

Recall that for a max-heap, every parent node must be greater than or equal to its children.

Formally:

$$A[\text{PARENT}(i)] \geq A[i] \quad \text{for all } i > 1$$

We can compute the indices of the left and right children as:

$$\text{LEFT}(i) = 2i, \quad \text{RIGHT}(i) = 2i + 1$$

Now, we check each parent node:

i	$A[i]$	LEFT(i)	RIGHT(i)	Children values	Condition
1	33	2	3	(19, 20)	$33 \geq 19, 20$ ✓
2	19	4	5	(15, 13)	$19 \geq 15, 13$ ✓
3	20	6	7	(10, 2)	$20 \geq 10, 2$ ✓
4	15	8	9	(13, 16)	$15 < 16$ ×
5	13	10	–	(12)	$13 \geq 12$ ✓

Since node 4 ($A[4] = 15$) is smaller than its right child ($A[9] = 16$), the max-heap property is violated.

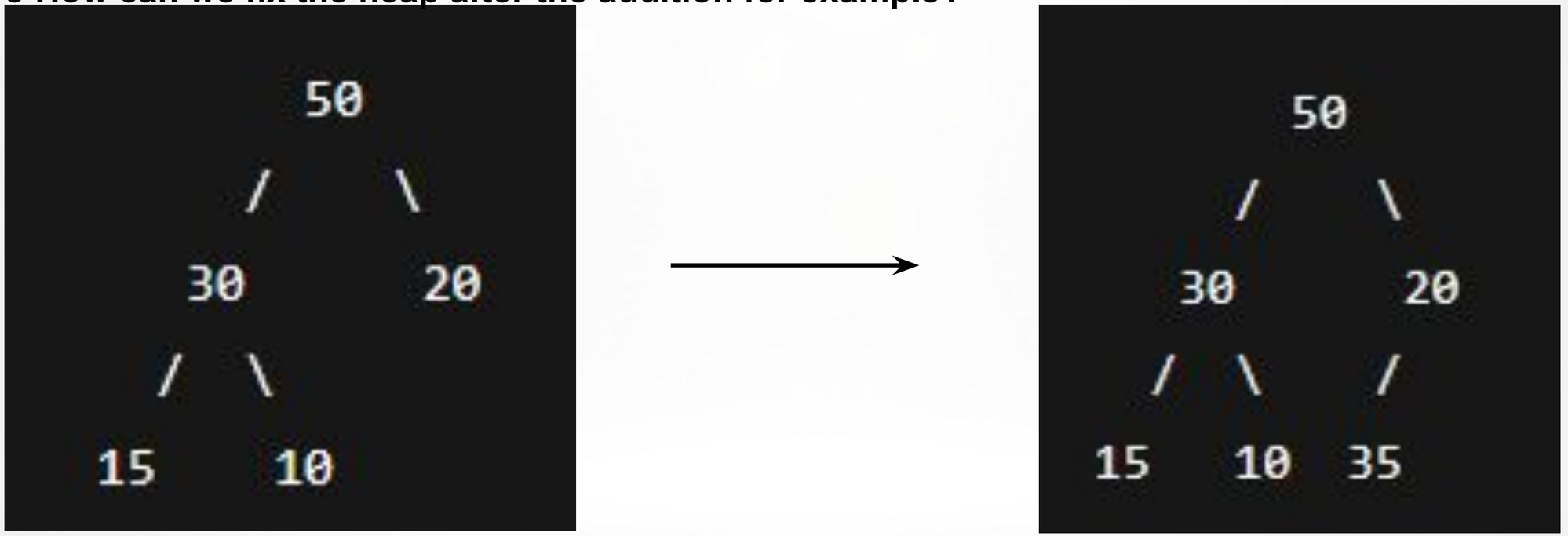
Therefore, the array is not a max-heap.

How Heaps Stay Organized ?

- The heap property must hold at all times:
  - For max-heap: every parent ≥ its children
  - For min-heap: every parent ≤ its children
- **When we insert, remove, or update elements, this property may break.**
- Heap operations are designed to restore this property efficiently.
- These operations make heaps suitable for:
  - Priority queues
  - Heap sort

Example – Adding element to a heap

- What Happens When We Add an element to a Heap?
- Start with a valid heap ⟨50, 30, 20, 15, 10⟩
- insert a new value to it ~ ***35***
  - ⟨50, 30, 20, 15, 10, 35⟩ → Not valid
- **So How can we fix the heap after the addition for example?**



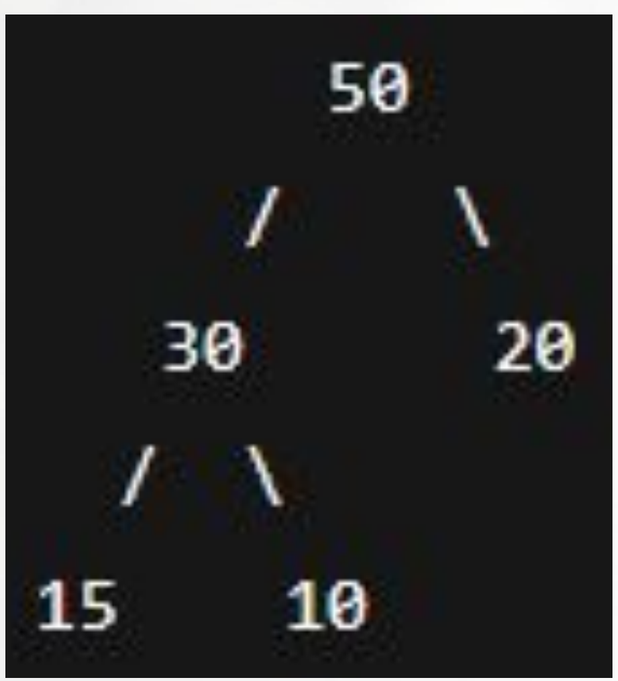
HEAP OPERATIONS

UpHeap (Bubble Up)

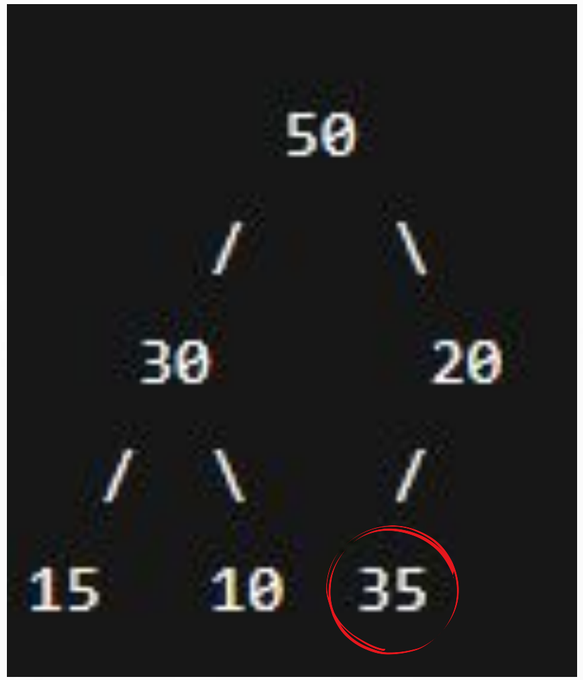
- Purpose:
  - Restore the heap property when a value increases (or a new element is inserted at the end).
- Idea:
  - Compare the node with its parent and swap upward until the heap property is restored.
- Time complexity: ***O(log n)***

```
UPHEAP(A, i)
    while i > 1 and A[PARENT(i)] < A[i]
        swap A[i] with A[PARENT(i)]
        i ← PARENT(i)
```

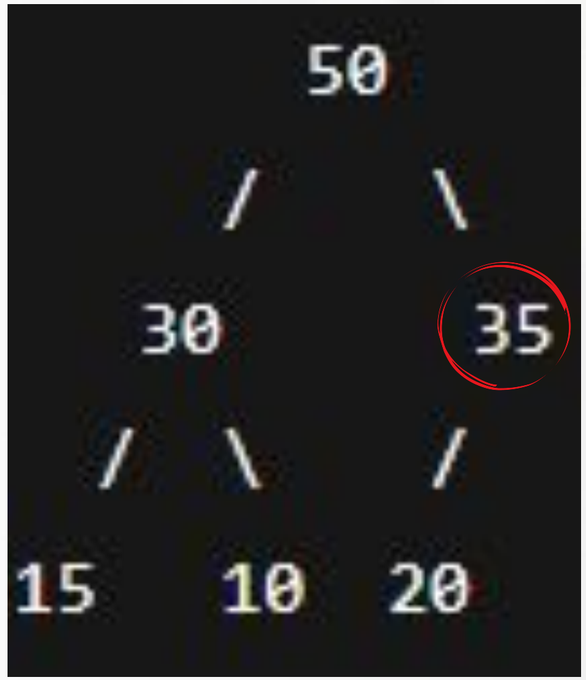
UpHeap (Bubble Up) – Example



Valid Max Heap



Add **35** – Invalid Max Heap



UpHeap Until it is valid

DownHeap (Heapify / Sift Down)

- Purpose:
  - Restore the heap property when a value decreases or after removing the root.
- Idea:
  - Compare a node with its children and swap downward with the larger child until the heap property holds.
- Time complexity: ***O(log n)***

```
DOWNHEAP(A, i, n)
    l ← LEFT(i)
    r ← RIGHT(i)
    largest ← i

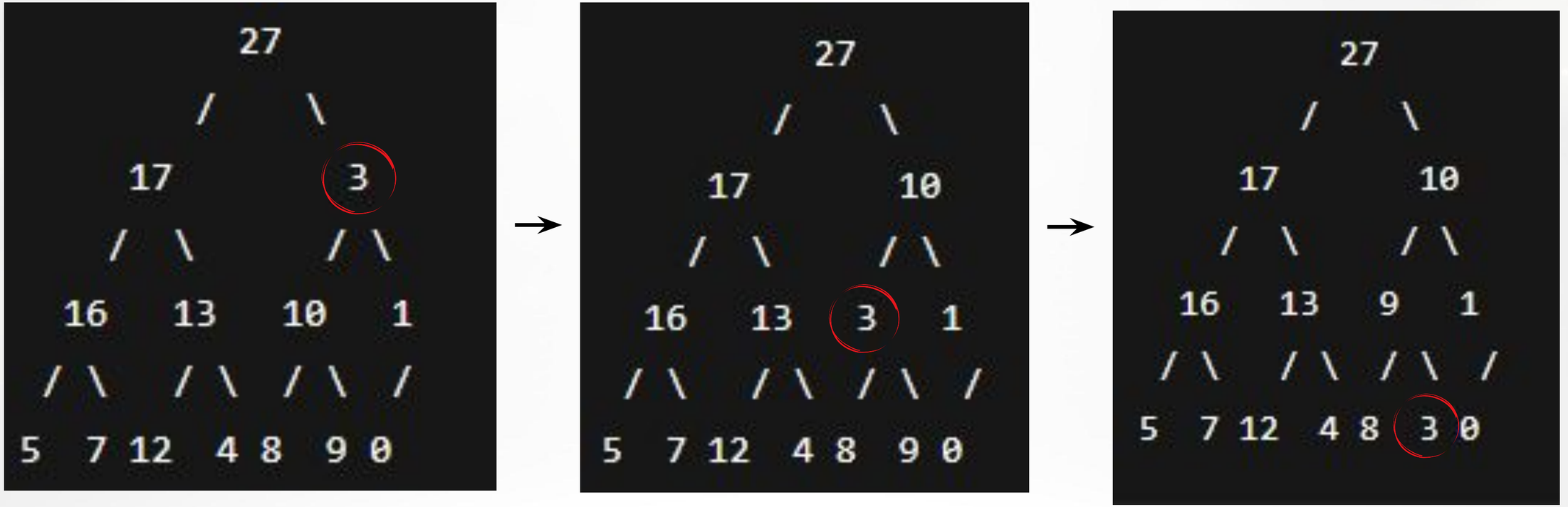
    if l ≤ n and A[l] > A[largest]
        largest ← l
    if r ≤ n and A[r] > A[largest]
        largest ← r

    if largest ≠ i
        swap A[i] with A[largest]
        DOWNHEAP(A, largest, n)
```


3

Sheet 5 – Question 2

- Illustrate the operation of DownHeap(A, 3) (or MAXHEAPIFY(A, 3)) on the array
  - A = ⟨27, 17, 3, 16, 13, 10, 1, 5, 7, 12, 4, 8, 9, 0⟩



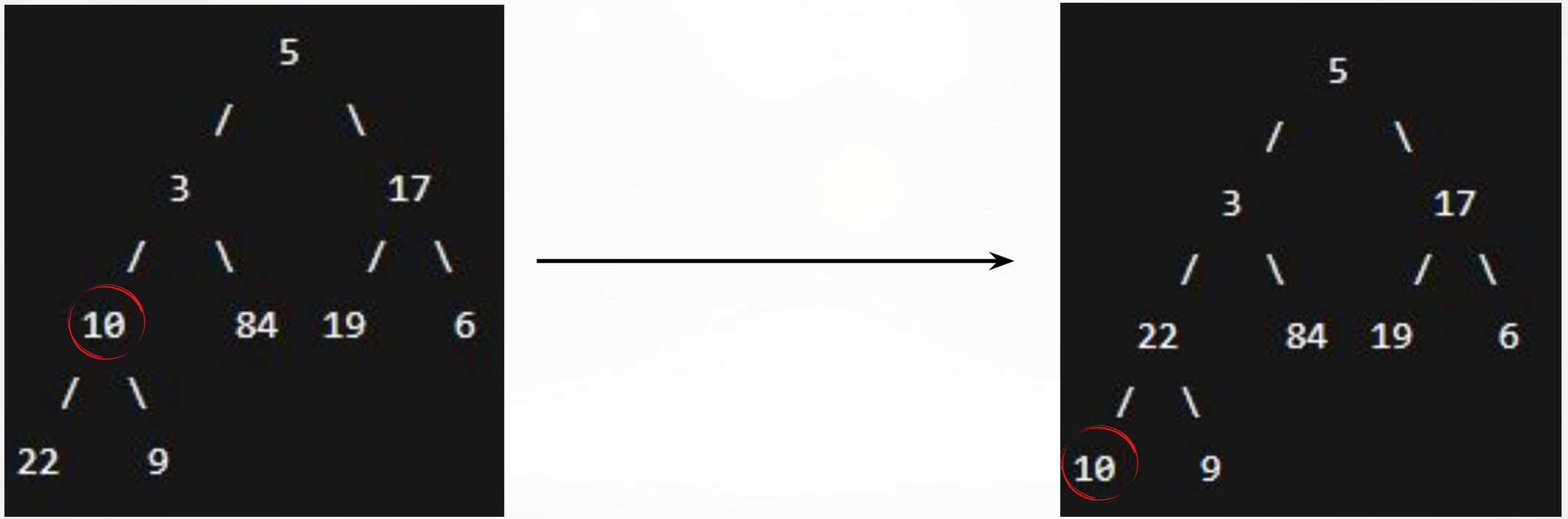
Build-Max-Heap

- Purpose:
  - Convert an unordered array into a valid max-heap.
- Idea:
  - Call DOWNHEAP on all internal nodes starting from ⌊n/2⌋ down to 1.
- Time complexity: ***O(n)***

```
BUILD-MAX-HEAP(A)
    n ← length(A)
    for i ← FLOOR(n / 2) downto 1
        DOWNHEAP(A, i, n)
```

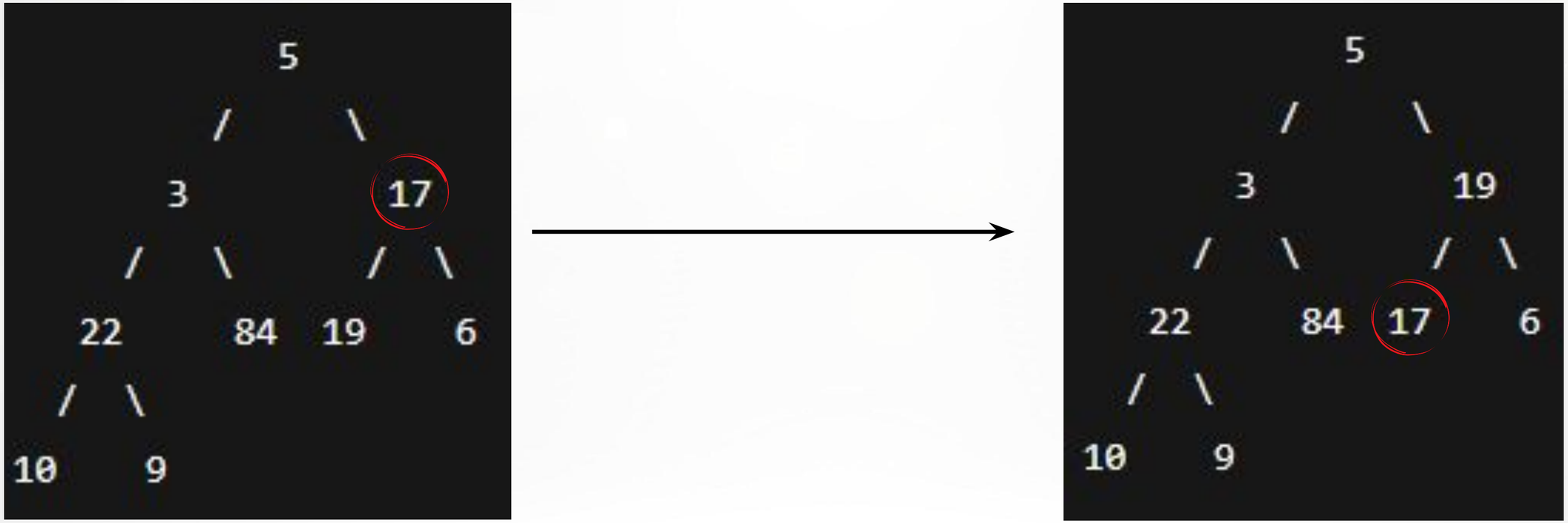
Sheet 5 – Question 3

- Illustrate the operation of BUILD-MAXHEAP on the array
  - A = ⟨5, 3, 17, 10, 84, 19, 6, 22, 9⟩
- Get floor(n/2) → 4 & arr[4] = 10 → (the book starts from 1)
- Now we iterate from 4 to 1 and each time we call DownHeap
- Step 1: **DownHeap/MaxHeapify at i = 4**



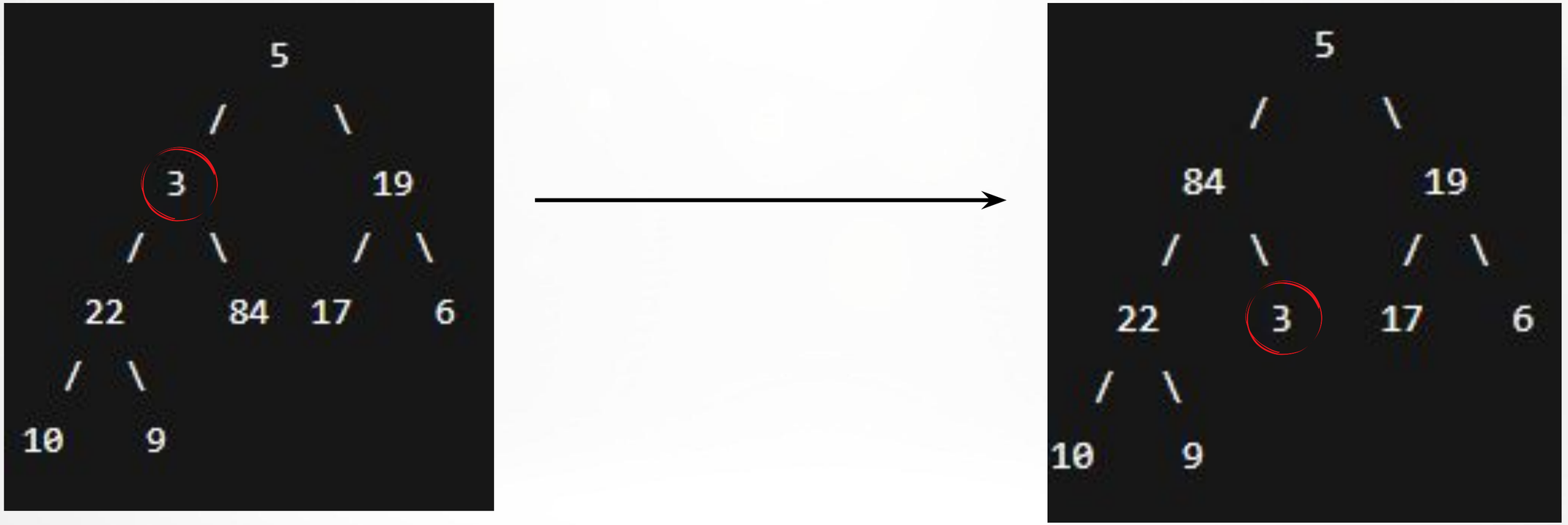
Sheet 5 – Question 3

- Illustrate the operation of BUILD-MAXHEAP on the array
  - A = ⟨5, 3, 17, 10, 84, 19, 6, 22, 9⟩
- Step 2: DownHeap/MaxHeapify at i = 3



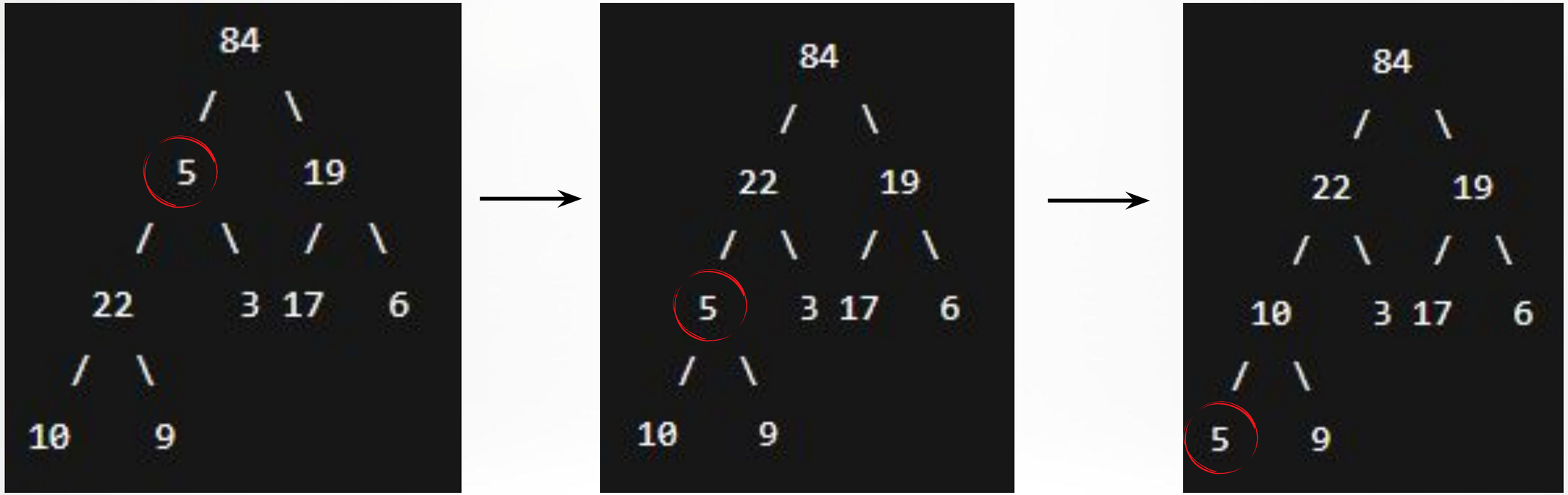
Sheet 5 – Question 3

- Illustrate the operation of BUILD-MAXHEAP on the array
  - A = ⟨5, 3, 17, 10, 84, 19, 6, 22, 9⟩
- Step 3: DownHeap/MaxHeapify at i = 2



Sheet 5 – Question 3

- Illustrate the operation of BUILD-MAXHEAP on the array
  - A = ⟨5, 3, 17, 10, 84, 19, 6, 22, 9⟩
- Step 3: DownHeap/MaxHeapify at i = 1



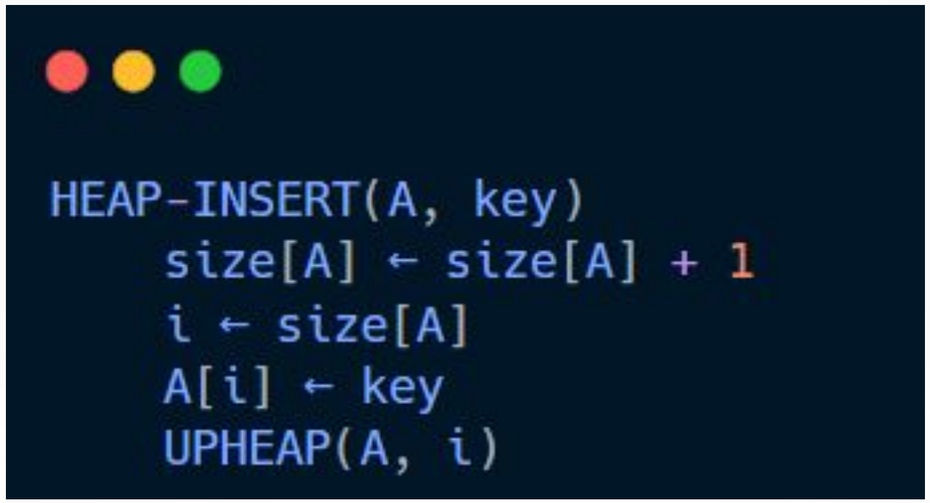
DeleteMax (ExtractMax)

- Purpose:
  - Remove and return the largest element from a max-heap while maintaining the heap property.
- Idea:
  - Swap the root (maximum) with the last element in the heap.
  - Reduce the heap size by 1.
  - Call DownHeap on the root to restore the max-heap property.
- Time complexity: ***O(log n)*** — because DownHeap may move down the height of the heap.

```
DELETE-MAX(A)
    if size[A] = 0 then
        return error
    MaxItem ← A[1]
    A[1] ← A[size[A]]
    size[A] ← size[A] - 1
    DOWNHEAP(A, 1)
    return MaxItem
```

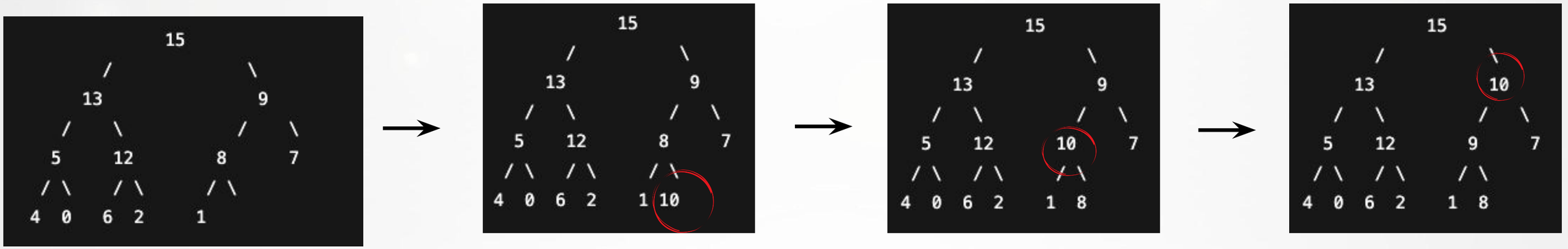
Heap-Insert (Insert a New Element into a Max-Heap)

- Purpose:
  - Add a new element to a max-heap while maintaining the heap property.
- Idea:
  - Insert the new element at the end of the heap.
  - “Bubble up” (UpHeap) the element along the ancestral path until the heap property is restored.
- Time complexity: ***O(log n)*** — because the new element may move up the height of the heap.



Sheet 5 – Question 6

- s. Illustrate the operation of MAX-HEAP-INSERT(A,10) on the heap
  - A = ⟨15, 13, 9, 5, 12, 8, 7, 4, 0, 6, 2, 1⟩

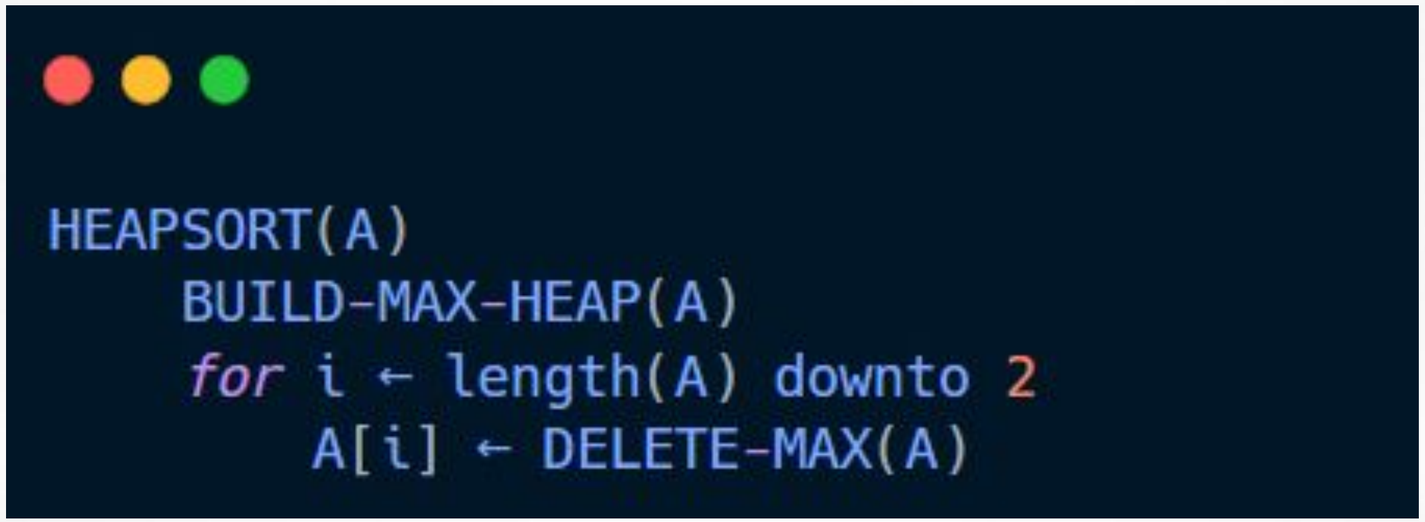


HEAP SORT

4

HeapSort

- Purpose:
  - Sort an array in ascending order using the max-heap structure.
- Idea:
  - Build a max-heap from the array.
  - Repeatedly remove the maximum element (DeleteMax) and place it at the end of the array.
- Time complexity:
  - ***O(n log n)*** — ***O(n)*** to build the heap + ***O(log n)*** per delete, repeated n times.



```
HEAPSORT(A)
    BUILD-MAX-HEAP(A)
    for i ← length(A) downto 2
        A[i] ← DELETE-MAX(A)
```

Sheet 5 – Question 5

- Illustrate the operation of HEAPSORT on the array
  - A = ⟨5, 13, 2, 25, 7, 17, 20, 8, 4⟩

Initial Array

A = ⟨5, 13, 2, 25, 7, 17, 20, 8, 4⟩

Step 1 — Build Max Heap

A = ⟨25, 13, 20, 8, 7, 17, 2, 5, 4⟩

Step 2 — HEAPSORT Process

Step	Swap Max with Last	After Down-Heap (Heapify)
1	⟨4, 13, 20, 8, 7, 17, 2, 5, **25**⟩	⟨20, 13, 17, 8, 7, 4, 2, 5, 25⟩
2	⟨5, 13, 17, 8, 7, 4, 2, **20**, 25⟩	⟨17, 13, 5, 8, 7, 4, 2, 20, 25⟩
3	⟨2, 13, 5, 8, 7, 4, **17**, 20, 25⟩	⟨13, 8, 5, 2, 7, 4, 17, 20, 25⟩
4	⟨4, 8, 5, 2, 7, **13**, 17, 20, 25⟩	⟨8, 7, 5, 2, 4, 13, 17, 20, 25⟩
5	⟨4, 7, 5, 2, **8**, 13, 17, 20, 25⟩	⟨7, 4, 5, 2, 8, 13, 17, 20, 25⟩
6	⟨2, 4, 5, **7**, 8, 13, 17, 20, 25⟩	⟨5, 4, 2, 7, 8, 13, 17, 20, 25⟩
7	⟨2, 4, **5**, 7, 8, 13, 17, 20, 25⟩	⟨4, 2, 5, 7, 8, 13, 17, 20, 25⟩
8	⟨2, **4**, 5, 7, 8, 13, 17, 20, 25⟩	⟨2, 4, 5, 7, 8, 13, 17, 20, 25⟩

Summarize Everything we saw

Algorithm	Best Case	Average Case	Worst Case
Selection Sort	$O(n^2)$	$O(n^2)$	$O(n^2)$
Insertion Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Bubble Sort	$O(n)$	$O(n^2)$	$O(n^2)$
Quick Sort	$O(n \log n)$	$O(n \log n)$	$O(n^2)$
Heap Sort	$O(n \log n)$	$O(n \log n)$	$O(n \log n)$

- Even with the best comparison-based algorithms, the worst-case complexity for sorting is O(nlogn).
- Can we do better?
- Spoiler: yes — but only in certain special cases.

LINEAR TIME O(N) SORTING

AND

COUNTING SORT

Linear Time Sorting O(n)

- Most comparison-based sorting algorithms cannot beat O(nlogn) in the worst case.
- Linear-time sorting is possible if:
  - The keys are integers or can be mapped to integers.
  - The range of keys is limited and known in advance.
- Examples:
  - Counting Sort
  - Radix Sort
  - Bucket Sort
- **Key Idea:** Use the values of the keys directly to determine their position rather than comparing them pairwise.

Counting Sort – Overview

- Non-comparison-based sorting algorithm.
  - Works for integers in a known range 0..k.
- Sorts in O(n+k) time.
- High-level idea:
  - Count occurrences of each key.
  - Compute `**cumulative counts**` to determine final positions.
  - Place elements into the output array using the counts.
- Wait what `**cumulative counts**`**?**

Before we continue – Cumulative Sum (Prefix Sum)

- Purpose:
  - Compute a running total of an array to determine positions or ranges efficiently.
- Example:
  - Given an array of counts:
    - counts = **[2, 1, 3, 0]**
  - Cumulative sum (prefix sum) array:
    - cumulative = **[2, 3, 6, 6]**
- **We use this as a step in the counting sort algorithm**

Counting Sort – Steps

- Purpose:
  - Sort an array of integers in linear time when keys are in a limited range.
- Idea:
  - Count how many times each key appears.
  - Compute the cumulative sum of the counts — this tells you the final position of each key in the sorted array.
  - Iterate over the original array from right to left, and for each element:
    - Use the cumulative count to determine its exact position in the output array.
    - Decrease the count so the next identical element goes to the previous available position.
- Time Complexity: ***O(n+k)***
- Space Complexity: ***O(n+k)***

```
COUNTING_SORT(inputArray, outputArray, maxValue)
    // Step 1: Initialize the count array

    for i ← 0 to maxValue do
        count[i] ← 0

    // Step 2: Count occurrences of each key

    for j ← 1 to length(inputArray) do
        count[inputArray[j]] ← count[inputArray[j]] + 1

    // Step 3: Compute cumulative counts

    for i ← 1 to maxValue do
        count[i] ← count[i] + count[i - 1]

    // Step 4: Build the output array

    for j ← length(inputArray) downto 1 do
        outputArray[count[inputArray[j]]] ← inputArray[j]
        count[inputArray[j]] ← count[inputArray[j]] - 1
```

Sheet 5 – Question 8

- Illustrate the operation of COUNTING SORT on the array
  - A = ⟨6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2⟩

Step 1: Count occurrences of each key:

Count array: $C[0..6] = [0, 0, 0, 0, 0, 0, 0]$

Increment counts:

$$
\begin{aligned}
A[1] = 6 &\rightarrow C[6] = 1 \\
A[2] = 0 &\rightarrow C[0] = 1 \\
A[3] = 2 &\rightarrow C[2] = 1 \\
A[4] = 0 &\rightarrow C[0] = 2 \\
A[5] = 1 &\rightarrow C[1] = 1 \\
A[6] = 3 &\rightarrow C[3] = 1 \\
A[7] = 4 &\rightarrow C[4] = 1 \\
A[8] = 6 &\rightarrow C[6] = 2 \\
A[9] = 1 &\rightarrow C[1] = 2 \\
A[10] = 3 &\rightarrow C[3] = 2 \\
A[11] = 2 &\rightarrow C[2] = 2
\end{aligned}
$$

Final counts: $C = [2, 2, 2, 2, 1, 0, 2]$

Step 2: Compute cumulative counts:

$$C = [2, 4, 6, 8, 9, 9, 11]$$

5

Sheet 5 – Question 8

- Illustrate the operation of COUNTING SORT on the array
  - A = ⟨6, 0, 2, 0, 1, 3, 4, 6, 1, 3, 2⟩

Step 3: Place elements into output array (iterate from right to left):

$$
\begin{aligned}
A[11] = 2 \rightarrow B[6] = 2 \quad C[2] &= 5 \\
A[10] = 3 \rightarrow B[8] = 3 \quad C[3] &= 7 \\
A[9] = 1 \rightarrow B[4] = 1 \quad C[1] &= 3 \\
A[8] = 6 \rightarrow B[11] = 6 \quad C[6] &= 10 \\
A[7] = 4 \rightarrow B[9] = 4 \quad C[4] &= 8 \\
A[6] = 3 \rightarrow B[7] = 3 \quad C[3] &= 6 \\
A[5] = 1 \rightarrow B[3] = 1 \quad C[1] &= 2 \\
A[4] = 0 \rightarrow B[2] = 0 \quad C[0] &= 1 \\
A[3] = 2 \rightarrow B[5] = 2 \quad C[2] &= 4 \\
A[2] = 0 \rightarrow B[1] = 0 \quad C[0] &= 0 \\
A[1] = 6 \rightarrow B[10] = 6 \quad C[6] &= 9
\end{aligned}
$$

Sorted array: $\langle 0, 0, 1, 1, 2, 2, 3, 3, 4, 6, 6 \rangle$

THANK YOU

By: Abdullah muhammed